Lab	Type	Practical
1	A	**Analyze and Design a Database Schema Using Logical Thinking (Leave Management System)** Design and implement a relational database for a Leave Management System. Create ORM/model classes for each entity, define relationships and constraints, and apply migrations so the schema is ready for use.
2	A	**Create RESTful API endpoints based on Leave Management System database schema.** Implement full CRUD functionality for all major entities—such as Employees, Leave Types, Leave Balances, Leave Requests, and Approval History
3	A	**Design user interface screens that will consume and interact with the developed APIs.** Create intuitive UI screens that allow users to perform actions such as applying for leave, viewing leave history, managing leave balances, and approving requests.
4	A	**Consume API.** Consume Leave Management System REST APIs by integrating them into the UI to fetch, create, update, and delete Leave Management data, ensuring proper handling of responses, errors, and loading states.
5	A	**Microservices Concepts & Baseline .NET API Architecture** Identify the main modules of a Leave Management System—such as Employee, Leave Request, and Leave Type—and separate them into individual microservices. Create a basic folder structure for each microservice, showing that each one works independently and has its own responsibility. Prepare a simple startup file or basic endpoint for at least one service to demonstrate how these services can run separately.
6	A	**Analyze Domains and Define Microservice Boundaries with ER Models** Identify the main parts of the Leave Management System (like Employees, Leave Requests, Leave Types) and group related features into separate microservices. Draw a simple ER diagram for each microservice showing its entities and how they are connected.

7	**A**	**Develop a Basic .NET Microservice with CRUD Operations** Create one microservice for a Leave Management feature (such as Leave Type or Employee) and build simple Create, Read, Update, and Delete API operations for it. Test each operation to ensure the microservice can add, fetch, modify, and remove data correctly.
8	**A**	**Implement Layered Architecture Using Controllers, Services, Repositories & DTOs** Organize microservice into clear layers by separating API controllers, business logic (services), data access (repositories), and data transfer objects (DTOs).
9	**A**	**Configure and Evaluate API Gateway Routing Using YARP/Ocelot** Set up an API Gateway and configure routes that forward requests to different microservices in the Leave Management System. Test a few routes to confirm that the gateway correctly receives incoming requests and passes them to the right service.
10	**A**	**Compare Sync vs Async Communication and Implement Basic Event Handling** Explain the difference between synchronous (direct API calls) and asynchronous (event-based) communication using simple Leave Management examples. Implement a basic event—such as "Leave Request Created"—and show how another service can receive and react to that event.
11	**A**	**Apply Dapper for CRUD Operations with SQL Fundamentals** Use Dapper to perform simple Create, Read, Update, and Delete operations for different modules of the Leave Management System. Write basic SQL queries for each operation and test them to ensure data is correctly stored and retrieved from the database.
12	**A**	**Develop and Execute Stored Procedures Using Dapper** Create stored procedures for key operations—such as inserting, updating, or retrieving Leave Management data—and save them in the database. Use Dapper to call these stored procedures from your application and verify that they execute correctly and return the expected results.

13	A	**Implement the Repository Pattern with Dapper and Dependency Injection** Convert your existing data-access logic to use Dapper inside a properly structured repository class for one chosen Leave Management module. Register this repository with Dependency Injection so it can be cleanly used by your service or controller without directly writing SQL in the API layer.
14	A	**Apply Memory and Redis Caching Strategies in .NET Applications** Implement basic in-memory caching for frequently accessed Leave Management data (such as Leave Types or Employee details) to reduce repeated database calls. Then set up Redis caching and update your code to store and retrieve the same data from Redis, comparing how both caching methods improve performance.
15	A	**Integrate Structured Logging Using Serilog/NLog in APIs** Add structured logging to your Leave Management microservice using Serilog or NLog, ensuring key actions (like creating or updating a leave request) are logged with clear details. Test your API calls to verify that logs are generated in a readable, structured format in the console or log file.
16	A	**Implement JWT Authentication and Validate Token-Based Access** Add JWT authentication to the Leave Management microservice so users must provide a valid token to access protected endpoints. Configure the API to generate tokens for valid users and verify incoming tokens before allowing any operation.
17	A	**Apply Role-Based Authorization to Secure API Endpoints** Assign roles such as Employee, Manager, or Admin within the Leave Management microservice and restrict specific API endpoints based on these roles.
18	A	**Implement and Test API Rate Limiting Strategies** Configure rate limiting in the Leave Management microservice to control how many requests a user can make within a set time period. Test the limits by sending multiple API calls and verify that the system blocks excessive requests while allowing normal usage.
19	A	**Create Unit Tests Using xUnit and Moq for Service Layer Validation** Write unit tests for the Leave Management service layer using xUnit and use Moq to mock the repository or external dependencies. Validate service methods.

20	**A**	**Perform Integration Testing for API, Database, and Auth Workflows** Create integration tests that call the actual API endpoints of your Leave Management microservice and verify interactions with the real database. Include authentication in your tests to ensure token validation, protected routes, and end-to-end workflows function correctly.
21	**A**	**Analyze Cloud Platforms and Deploy a .NET API to the Cloud** Deploy one microservice from the Leave Management System to a cloud platform such as Azure, AWS, or Google Cloud. Ensure the API runs successfully online and can be accessed through its cloud-hosted endpoint.
22	**A**	**Configure and Test Cloud SQL Databases (Azure/AWS) with Microservices** Create a cloud-based SQL database on Azure or AWS and connect one Leave Management microservice to it. Run basic CRUD operations to confirm that the microservice successfully reads and writes data to the cloud database.
23	**A**	**Implement Cloud Storage Integration for File Upload and Access** Connect your Leave Management microservice to a cloud storage service (Azure Blob Storage or AWS S3) and upload a sample file such as a leave attachment or document. Retrieve and access the uploaded file using a storage URL to confirm that the integration works correctly.
24	**A**	**Develop Real-Time Communication Using SignalR Hubs** Create a simple SignalR hub in your Leave Management microservice that sends real-time notifications (e.g., “New Leave Request Submitted”). Build a small client page or console app to connect to the hub and display these notifications instantly as they are triggered.
25	**A**	**Implement Group Messaging Using SignalR and Explore .NET Aspire** Set up a SignalR hub that allows sending messages to specific groups, such as notifying only managers when a leave request is submitted. Then briefly explore .NET Aspire by running a sample Aspire dashboard or project to understand how it helps monitor and manage distributed applications.

26	**A**	**Background Jobs, Queuing & Messaging** Set up a simple background job processor (using Hangfire, Quartz, or a message queue like RabbitMQ) to handle tasks such as sending leave approval emails or updating leave balances in the background. Push a sample message or job into the queue and verify that the background worker processes it without blocking the main API.
27	**A**	**Develop RabbitMQ Producer–Consumer Messaging Services** Create a RabbitMQ producer in one microservice that publishes a message when a new leave request is created (e.g., "LeaveRequestCreated"). Build a separate consumer service that receives this message and performs a background action, such as calculating the total leave days or updating the employee's remaining leave balance.
28	**A**	**Implement Publish–Subscribe Event-Driven Communication Using Topic Exchanges** Set up a topic exchange in RabbitMQ and publish an event—for example, "LeaveRequest.Updated"—from one service. Create two or more subscribers that listen to different topic patterns and confirm they each receive the event based on their subscribed topic rules.
29	**A**	**Simulate the Saga Pattern with Distributed Transaction Logic** Create two simple microservices (e.g., Leave Request Service and Leave Balance Service) and simulate a distributed transaction where a leave request is created only if the employee has enough leave balance. Use a sequence of events to mimic the Saga flow and show how one service can compensate (e.g., cancel the request) if another service reports a failure.
30	**A**	**Build an End-to-End Event-Driven Microservice Workflow** Create an event-driven workflow where the Leave Request Service publishes an event such as "LeaveRequest.Submitted", and multiple other services react to it—e.g., the Leave Balance Service checks available balance and the Audit Service logs the action. Verify that each service processes the event independently, creating a full end-to-end workflow without direct service-to-service calls.